Object-Oriented Programming Concepts in Java

1. Classes and Objects

What is a Class?

A class is a blueprint or template that defines the structure and behavior of objects. It contains attributes (fields) and methods that describe what an object can do.

What is an Object?

An object is an instance of a class. It's a real-world entity that has state (attributes) and behavior (methods).

Creating Classes and Objects

```java
// Class definition
public class Student {
    // Attributes (instance variables)
    private String name;
    private int age;
    private String studentId;

    // Constructor
    public Student(String name, int age, String studentId) {
        this.name = name;
        this.age = age;
        this.studentId = studentId;
    }

    // Method
    public void displayInfo() {
        System.out.println("Name: " + name + ", Age: " + age + ", ID: " + studentId);
    }

    // Getter methods
    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }
}

// Creating and using objects
public class Main {
    public static void main(String[] args) {
        // Creating objects
        Student student1 = new Student("Alice", 20, "S001");
        Student student2 = new Student("Bob", 19, "S002");

        // Using objects
        student1.displayInfo();
        student2.displayInfo();
    }
}
```

2. Encapsulation

What is Encapsulation?

Encapsulation is the practice of hiding internal implementation details and exposing only necessary information through public methods. It's achieved using access modifiers.

Access Modifiers

- **private**: Accessible only within the same class
- **protected**: Accessible within the same package and subclasses
- **public**: Accessible from anywhere
- **default** (no modifier): Accessible within the same package

Example of Encapsulation

```java
public class BankAccount {
    private double balance; // Private field - encapsulated
    private String accountNumber;

    public BankAccount(String accountNumber, double initialBalance) {
        this.accountNumber = accountNumber;
        this.balance = initialBalance;
    }

    // Public methods to access private data
    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited: $" + amount);
        }
    }

    public boolean withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawn: $" + amount);
            return true;
        }
        return false;
    }
}
```

3. Inheritance

What is Inheritance?

Inheritance allows a class to inherit properties and methods from another class. The class that inherits is called a subclass (child), and the class being inherited from is called a superclass (parent).

Types of Inheritance in Java

- **Single Inheritance**: One class inherits from one superclass
- **Multilevel Inheritance**: A chain of inheritance
- **Hierarchical Inheritance**: Multiple classes inherit from one superclass

Inheritance Example

java

```java
// Parent class
public class Animal {
    protected String name;
    protected int age;

    public Animal(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void eat() {
        System.out.println(name + " is eating");
    }

    public void sleep() {
        System.out.println(name + " is sleeping");
    }
}

// Child class
public class Dog extends Animal {
    private String breed;

    public Dog(String name, int age, String breed) {
        super(name, age); // Call parent constructor
        this.breed = breed;
    }

    public void bark() {
        System.out.println(name + " is barking");
    }

    // Method overriding
    @Override
    public void eat() {
        System.out.println(name + " the dog is eating dog food");
    }
}

// Usage
public class TestInheritance {
    public static void main(String[] args) {
        Dog myDog = new Dog("Buddy", 3, "Golden Retriever");
        myDog.eat();    // Overridden method
        myDog.sleep();  // Inherited method
        myDog.bark();   // Dog-specific method
```

```
    }
}
```

4. Polymorphism

What is Polymorphism?

Polymorphism means "many forms." It allows objects of different classes to be treated as objects of a common base class while maintaining their specific behaviors.

Types of Polymorphism

1. **Compile-time Polymorphism** (Method Overloading)
2. **Runtime Polymorphism** (Method Overriding)

Method Overloading Example

```java
public class Calculator {
    // Same method name, different parameters
    public int add(int a, int b) {
        return a + b;
    }

    public double add(double a, double b) {
        return a + b;
    }

    public int add(int a, int b, int c) {
        return a + b + c;
    }
}
```

Method Overriding and Runtime Polymorphism

java

```java
// Base class
abstract class Shape {
    public abstract double calculateArea();

    public void display() {
        System.out.println("This is a shape");
    }
}

// Derived classes
class Circle extends Shape {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    @Override
    public double calculateArea() {
        return Math.PI * radius * radius;
    }
}

class Rectangle extends Shape {
    private double length, width;

    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    @Override
    public double calculateArea() {
        return length * width;
    }
}

// Polymorphism in action
public class PolymorphismDemo {
    public static void main(String[] args) {
        Shape[] shapes = {
            new Circle(5),
            new Rectangle(4, 6)
        };

        for (Shape shape : shapes) {
```

```
            System.out.println("Area: " + shape.calculateArea());
        }
    }
}
```

5. Abstraction

What is Abstraction?

Abstraction hides complex implementation details and shows only essential features. It can be achieved through abstract classes and interfaces.

Abstract Classes

```java
abstract class Vehicle {
    protected String brand;
    protected int year;

    public Vehicle(String brand, int year) {
        this.brand = brand;
        this.year = year;
    }

    // Abstract method - must be implemented by subclasses
    public abstract void start();

    // Concrete method
    public void displayInfo() {
        System.out.println("Brand: " + brand + ", Year: " + year);
    }
}

class Car extends Vehicle {
    public Car(String brand, int year) {
        super(brand, year);
    }

    @Override
    public void start() {
        System.out.println("Car engine started with key");
    }
}

class Motorcycle extends Vehicle {
    public Motorcycle(String brand, int year) {
        super(brand, year);
    }

    @Override
    public void start() {
        System.out.println("Motorcycle started with kick/button");
    }
}
```

Interfaces

```java
interface Drawable {
    void draw(); // Abstract method (implicitly public and abstract)

    default void display() { // Default method (Java 8+)
        System.out.println("Displaying the drawable object");
    }
}

interface Colorable {
    void setColor(String color);
}

// A class can implement multiple interfaces
class Circle implements Drawable, Colorable {
    private String color;
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    @Override
    public void draw() {
        System.out.println("Drawing a circle with radius " + radius);
    }

    @Override
    public void setColor(String color) {
        this.color = color;
        System.out.println("Circle color set to " + color);
    }
}
```

6. Constructors

What are Constructors?

Constructors are special methods used to initialize objects when they are created. They have the same name as the class and no return type.

Types of Constructors

```java
public class Person {
    private String name;
    private int age;

    // Default constructor
    public Person() {
        this.name = "Unknown";
        this.age = 0;
    }

    // Parameterized constructor
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Copy constructor
    public Person(Person other) {
        this.name = other.name;
        this.age = other.age;
    }

    public void displayInfo() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}
```

7. Method Overloading vs Method Overriding

Method Overloading

- Same method name, different parameters
- Compile-time polymorphism
- Occurs within the same class

Method Overriding

- Same method signature in parent and child class
- Runtime polymorphism
- Requires inheritance

```java
class Parent {
    public void display() {
        System.out.println("Parent display");
    }

    public void show(int x) {
        System.out.println("Parent show: " + x);
    }
}

class Child extends Parent {
    // Method overriding
    @Override
    public void display() {
        System.out.println("Child display");
    }

    // Method overloading
    public void show(int x, int y) {
        System.out.println("Child show: " + x + ", " + y);
    }
}
```

Practice Session

Key Points to Remember:

1. **Encapsulation**: Use private fields with public getter/setter methods
2. **Inheritance**: Use `extends` keyword; call parent constructor with `super()`
3. **Polymorphism**: Same method name, different behavior
4. **Abstraction**: Use abstract classes and interfaces to hide implementation details
5. **Constructor**: Special method to initialize objects; same name as class

Common OOP Principles:

- **DRY (Don't Repeat Yourself)**: Use inheritance to avoid code duplication
- **Single Responsibility**: Each class should have one reason to change
- **Open/Closed Principle**: Classes should be open for extension, closed for modification

---

Quiz Questions

1. Which of the following best describes encapsulation in Java? a) The ability of a class to inherit properties from another class b) The practice of hiding internal implementation details and exposing only necessary information c) The ability to create multiple methods with the same name d) The process of creating abstract classes

Correct Answer: b) The practice of hiding internal implementation details and exposing only necessary information

2. What keyword is used to inherit from a class in Java? a) implements b) inherits c) extends d) super

Correct Answer: c) extends

3. Which type of polymorphism is achieved through method overriding? a) Compile-time polymorphism b) Runtime polymorphism c) Static polymorphism d) Dynamic binding polymorphism

Correct Answer: b) Runtime polymorphism

4. What is true about abstract classes in Java? a) They can be instantiated directly b) They cannot contain concrete methods c) They can contain both abstract and concrete methods d) They must contain only abstract methods

Correct Answer: c) They can contain both abstract and concrete methods

5. Which access modifier makes a field accessible only within the same class? a) public b) protected c) private d) default

Correct Answer: c) private

Practice Coding Questions

1. Library Management System Create a class hierarchy for a library management system:

- Create an abstract class `LibraryItem` with fields: title, author, itemId
- Create concrete classes `Book` and `Magazine` that extend `LibraryItem`
- `Book` should have additional field: numberOfPages
- `Magazine` should have additional field: issueNumber
- Implement appropriate constructors, getters, and a method `displayInfo()` in each class
- Create a `Library` class that can store an array of `LibraryItem` objects and has methods to add items and display all items

2. Shape Calculator Design a program using interfaces and polymorphism:

- Create an interface `Measurable` with methods `calculateArea()` and `calculatePerimeter()`
- Implement this interface in classes `Rectangle`, `Circle`, and `Triangle`

- Create a `ShapeCalculator` class with a method that accepts an array of `Measurable` objects and returns the total area of all shapes
- Demonstrate polymorphism by creating different shapes and calculating their total area

3. Employee Management System Create an employee management system using OOP concepts:

- Create a base class `Employee` with fields: name, id, baseSalary
- Create subclasses `Manager`, `Developer`, and `Intern` that extend `Employee`
- Each subclass should override a method `calculateSalary()` with different logic:
  - Manager: baseSalary + 20% bonus
  - Developer: baseSalary + overtime hours * hourly rate
  - Intern: fixed stipend amount
- Implement proper encapsulation with private fields and public methods
- Create a `Company` class that manages a list of employees and can calculate total payroll

4. Banking System Design a banking system demonstrating encapsulation and inheritance:

- Create an abstract class `Account` with fields: accountNumber, balance, accountHolder
- Create concrete classes `SavingsAccount` and `CheckingAccount`
- `SavingsAccount` should have an interest rate and method to calculate interest
- `CheckingAccount` should have an overdraft limit
- Implement methods: deposit(), withdraw(), getBalance()
- Ensure that balance cannot be directly accessed or modified (proper encapsulation)
- Handle edge cases like insufficient funds and negative deposits

5. University Course System Create a course management system using multiple OOP concepts:

- Create an interface `Enrollable` with methods `enroll(Student s)` and `getEnrolledCount()`
- Create classes `Student` with fields: name, studentId, courses enrolled
- Create class `Course` that implements `Enrollable` with fields: courseName, courseCode, instructor, maxCapacity
- Create class `Instructor` with fields: name, employeeId, courses teaching
- Implement a `University` class that manages students, courses, and instructors
- Demonstrate composition, aggregation, and proper use of interfaces